

Tarea Obligatoria, Redes de Datos, curso 1erS 2026.

Hybrid Cloud.

**Diagnóstico multi-capa de un pipeline de Machine Learning
sobre VPN site-to-site.**

Departamento de Informática.

Equipo I

Valentín Rodríguez, Martina Caetano, Mikaela Maldonado, Alexia Aurrecochea



Universidad Católica del Uruguay. Facultad de Ingeniería y Tecnologías.

Docentes: Ing. Wiler F. Alvez, Lic. Rodrigo Pastorín

28 de Junio, 2026

Introducción

El modelo de infraestructura híbrida surge como respuesta a las limitaciones de los entornos exclusivamente locales (*on-premises*) y de la nube pública. En una arquitectura *Hybrid Cloud* una organización opera simultáneamente recursos propios y servicios contratados a proveedores como AWS, Azure o Google Cloud, lo que impone desafíos de red concretos: conectividad segura entre entornos heterogéneos, baja latencia inter-sitio y visibilidad unificada del estado de la infraestructura distribuida.

Este trabajo investiga los aspectos de red de una arquitectura Hybrid Cloud a través de la construcción de una maqueta funcional de extremo a extremo. Sobre la maqueta se desplegó un *pipeline* de Machine Learning para detección de tráfico anómalo, cuyos datos residen en Azure Blob Storage y cuyo procesamiento se ejecuta on-premises. La interconexión entre ambos sitios se realiza mediante un túnel VPN site-to-site con WireGuard. Adicionalmente, se desplegaron dos clusters Kubernetes unidos por un *service mesh* multi-cluster con Istio para gestionar la comunicación entre microservicios distribuidos entre ambos sitios.

El foco del trabajo está en cómo se diagnostican y resuelven en vivo fallas de red sutiles sobre este escenario (en particular el problema clásico de PMTU black hole sobre VPNs), en el costo de overhead que introduce un service mesh frente a sus beneficios de conectividad y seguridad, y en cómo construir un observatorio multi-capa que permita ver el estado de las cinco capas OSI simultáneamente. El diseño del modelo de Machine Learning queda fuera del alcance: se utiliza únicamente como carga de trabajo realista.

Marco teórico y relación con el curso

El proyecto integra de forma práctica varias de las tecnologías y conceptos vistos a lo largo del curso:

Redes LAN y WAN. Cada cluster constituye una red local (LAN) con su propio rango de direccionamiento privado. La comunicación entre los dos equipos físicos representa un enlace WAN: en una arquitectura híbrida real, el datacenter on-premises y la nube están separados por Internet. Para simular el comportamiento de una WAN degradada (latencia y pérdida de paquetes) se utilizó la herramienta `tc netem` del kernel de Linux sobre la interfaz del túnel.

VPN (Virtual Private Network). La conexión entre los dos sitios se establece mediante WireGuard, un protocolo de VPN moderno incorporado al kernel de Linux. WireGuard utiliza UDP como transporte y criptografía Curve25519 sobre el Noise Protocol Framework. Crea un túnel cifrado punto a punto entre los dos equipos, sobre el cual viaja todo el tráfico inter-sitio. Conceptualmente es equivalente a Azure VPN Gateway o AWS Site-to-Site VPN, y resuelve el requisito fundamental de la hybrid cloud: extender la red privada de un entorno al otro de forma segura sobre un medio público.

PMTU y fragmentación IP. Cuando un paquete IP atraviesa un túnel, el header de encapsulación reduce la MTU efectiva del camino. Si el emisor no se entera de esa reducción (porque los mensajes ICMP *Fragmentation Needed* son bloqueados por algún equipo intermedio) se produce el fenómeno conocido como *PMTU black hole*: el TCP queda esperando ACKs que nunca llegan, sin error explícito. Los RFCs centrales para este fenómeno son: RFC 791 (Internet Protocol, flag DF), RFC 792

(ICMP), RFC 1191 (Path MTU Discovery), RFC 2923 (TCP Problems with PMTUD) y RFC 4821 (Packetization Layer PMTUD).

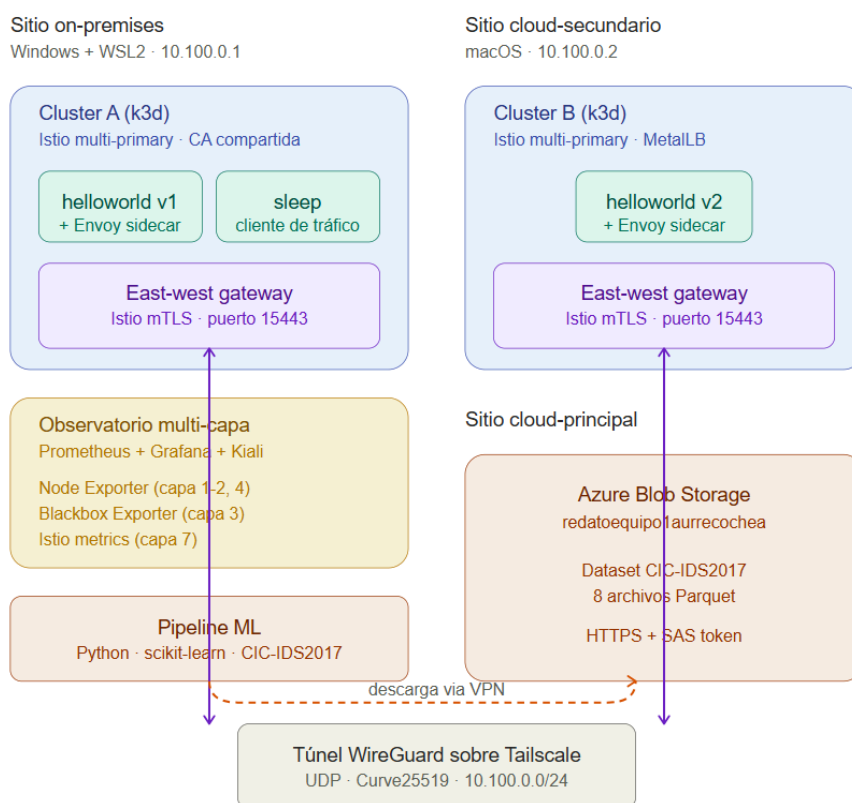
SDN (Software-Defined Networking). Un *service mesh* es una aplicación de los principios de SDN al tráfico entre microservicios. Istio separa el plano de control (el componente *istiod*, que decide cómo debe fluir el tráfico) del plano de datos (los proxies Envoy inyectados junto a cada servicio, que ejecutan esas decisiones). El operador define políticas de tráfico de forma declarativa y el mesh las propaga y aplica automáticamente, sin tocar el código de la aplicación.

APIs. Toda la configuración de la solución se realiza a través de APIs declarativas: la API de Kubernetes para desplegar workloads, la API de Istio (recursos *DestinationRule*, *VirtualService*, *Gateway*) para gestionar el tráfico, y la API REST de Azure Blob Storage para consumir los datos remotos.

Seguridad: mTLS. Istio establece *mutual TLS* automáticamente entre todos los proxies del mesh. Cada servicio se autentica con un certificado firmado por una autoridad certificadora (CA) común a ambos clusters, y todo el tráfico inter-servicio viaja cifrado. Esto es crítico en hybrid cloud, donde el tráfico cruza fronteras de red que pueden no ser confiables.

Arquitectura de la solución

La maqueta se compone de 3 componentes principales conectados por una VPN site-to-site:



- **Sitio on-premises** sobre un equipo Windows con WSL2 (Ubuntu 24.04). Aloja un cluster Kubernetes (k3d) con la versión v1 del servicio de prueba, un cliente generador de tráfico, el

pipeline de Machine Learning en Python con scikit-learn, y la capa de observabilidad completa: Prometheus para métricas, Grafana para dashboards, Kiali para visualización del grafo de servicios, y dos exporters adicionales (Node Exporter para métricas del sistema y Blackbox Exporter para probes ICMP).

- **Sitio cloud-secundario** sobre un equipo macOS, que aloja un segundo cluster Kubernetes (k3d) con la versión v2 del servicio de prueba. Este sitio simula el nodo de cómputo remoto y valida la conectividad bidireccional.
- **Sitio cloud-principal** en Azure Blob Storage, que aloja el dataset CIC-IDS2017 del Canadian Institute for Cybersecurity en formato Parquet particionado (ocho archivos correspondientes a distintos días y tipos de tráfico de red, incluyendo ataques DDoS, PortScan e Infiltración).

Interconexión. Los dos equipos físicos están conectados por un túnel WireGuard. Dado que las redes universitarias suelen aislar a los clientes entre sí (*client isolation*), el túnel se estableció sobre una capa de conectividad de Tailscale, que provee direcciones IP estables entre los pares independientemente de la red física subyacente. Tailscale utiliza WireGuard internamente, de modo que la solución sigue apoyándose en el mismo protocolo de VPN.

Service mesh. Sobre la conectividad de red se instaló Istio en modo *multi-primary*: ambos clusters tienen su propio plano de control *istiod*, y cada uno conoce los servicios del otro mediante un *east-west gateway* y un intercambio de *remote secrets*. El componente que hace posible el cifrado cross-cluster es la **CA compartida**: ambos clusters firman los certificados de sus servicios con la misma autoridad raíz.

Observatorio multi-capa. Un dashboard único de Grafana, alimentado por Prometheus, muestra simultáneamente métricas de las capas OSI relevantes con refresco de cinco segundos.

Implementación

La puesta en marcha siguió una secuencia incremental, validando cada capa antes de avanzar a la siguiente. Primero se levantó el túnel WireGuard entre los dos equipos, asignando direcciones del rango **10.100.0.0/24** a cada extremo. Se verificó la conectividad con **ping** y se confirmó el handshake con **wg show**. Cada cluster kubernetes se creó con rangos de direccionamiento distintos (**10.42.0.0/16** y **10.43.0.0/16** para el cluster A; **10.52.0.0/16** y **10.53.0.0/16** para el cluster B). Esta separación de CIDRs es un requisito de Istio multi-cluster: si los rangos se solaparan, el ruteo entre clusters sería ambiguo.

Luego se generó una autoridad raíz común con los scripts de Istio y se instaló en ambos clusters como el secret **cacerts** (CA compartida). Se instaló Istio en ambos clusters, se desplegaron los east-west gateways y se intercambiaron los *remote secrets*. Para que el endpoint del cluster remoto fuera alcanzable a través de Tailscale, se expuso el east-west gateway con una dirección IP accesible vía el túnel y se configuró un balanceador de carga (MetalLB) en el cluster cloud.

Se desarrolló un pipeline de machine learning en Python que se conecta a Azure Blob Storage mediante un SAS token, descarga los archivos Parquet del dataset CIC-IDS2017, los preprocesa con pandas y entrena un clasificador Random Forest con scikit-learn para detectar tráfico anómalo. La descarga atraviesa el túnel WireGuard, lo que permite medir el throughput real del enlace híbrido.

Se instaló Node Exporter (capa 1-2: métricas de interfaces, capa 4: TCP) y Blackbox Exporter (capa 3: probes ICMP) en el sitio on-premises. Se configuró Prometheus para *scrapear* ambos exporters junto con las métricas nativas de Istio (capa 7). Se construyó un dashboard de Grafana con ocho paneles, dos por capa OSI relevante.

Finalmente las pruebas consistieron en desplegar `helloworld` con la versión v1 en el cluster A y la v2 en el cluster B, y realizar peticiones repetidas desde un cliente en el cluster A. El resultado esperado (y obtenido) fue una mezcla de respuestas de ambas versiones, demostrando que el mesh balancea tráfico entre clusters de forma transparente:

14 Hello version: v1, instance: helloworld-v1-... (cluster A, on-prem)
6 Hello version: v2, instance: helloworld-v2-... (cluster B, cloud)

Metodología experimental

Se diseñaron tres experimentos para cuantificar y demostrar distintos aspectos de la maqueta:

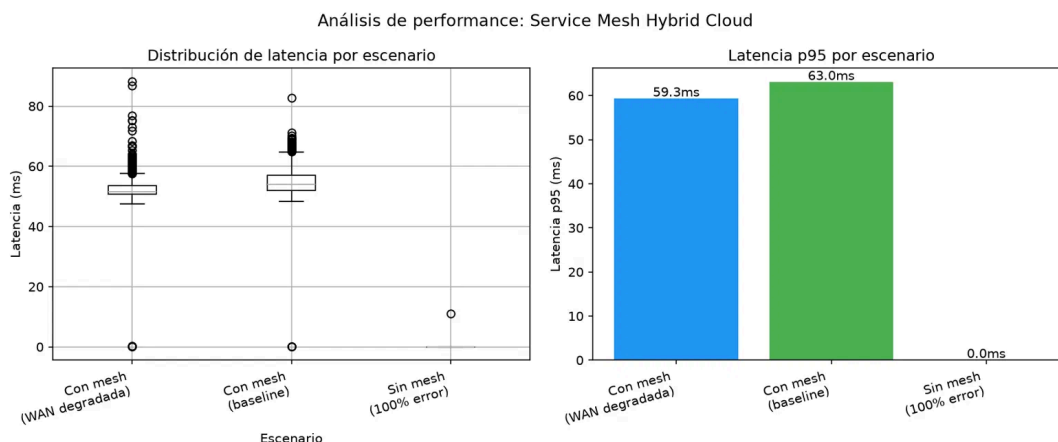
Experimento 1: Overhead del service mesh. Con la herramienta k6 se generó una tasa constante de 10 peticiones por segundo durante 2 minutos (1201 peticiones por corrida) contra el servicio `helloworld`. Se midió la latencia y la tasa de error en tres escenarios: (a) con mesh en condiciones normales, (b) con mesh y WAN degradada (≈ 150 ms RTT, 5 % de pérdida vía `tc netem`), y (c) sin mesh (retirando la inyección del sidecar).

Experimento 2: Pipeline ML sobre VPN. Se ejecutó el pipeline de detección de tráfico anómalo descargando el archivo `Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.parquet` (28.4 MB, 225 745 muestras) desde Azure Blob Storage a través del túnel WireGuard. Se midió el throughput de la descarga, el tiempo de entrenamiento y la *accuracy* del clasificador como prueba de que la carga de trabajo es funcionalmente realista.

Experimento 3: Diagnóstico PMTU black hole en vivo. Con el observatorio funcionando, se redujo la MTU del túnel WireGuard a 576 bytes y se bloquearon los mensajes ICMP `Fragmentation Needed` con iptables. Se generó tráfico sostenido a través del túnel y se observó la evolución de las métricas en las cuatro capas del dashboard. Luego se aplicó la mitigación estándar (MSS clamping mediante iptables) y se observó la recuperación.

Resultados

1. Service mesh: overhead y resiliencia



En operación nominal el service mesh tuvo una latencia p95 de 63 ms con 0 % de error. Bajo WAN degradada la latencia se mantuvo en 59 ms con 0 % de error: la degradación no impactó significativamente el tráfico observado. Sin mesh, el 100 % de las peticiones falló con error de conexión: al retirar el sidecar, el servicio dejó de ser alcanzable por la vía configurada.

```

python3 pipeline/ml_pipeline.py
=====
Pipeline ML - Detección de Tráfico Anómalo
Hybrid Cloud: Azure Blob Storage → on-prem via WireGuard
=====
[1/4] Conectando a Azure Blob Storage...
Account: redatoequipolaurrecochea
Container: cic-ids2017
Blob: Friday-WorkingHours-Afternoon-DDoS.pcap_ISCX.parquet
Descargando datos a través del túnel WireGuard...
✓ 28.4 MB descargados en 10.2s (2.8 MB/s)

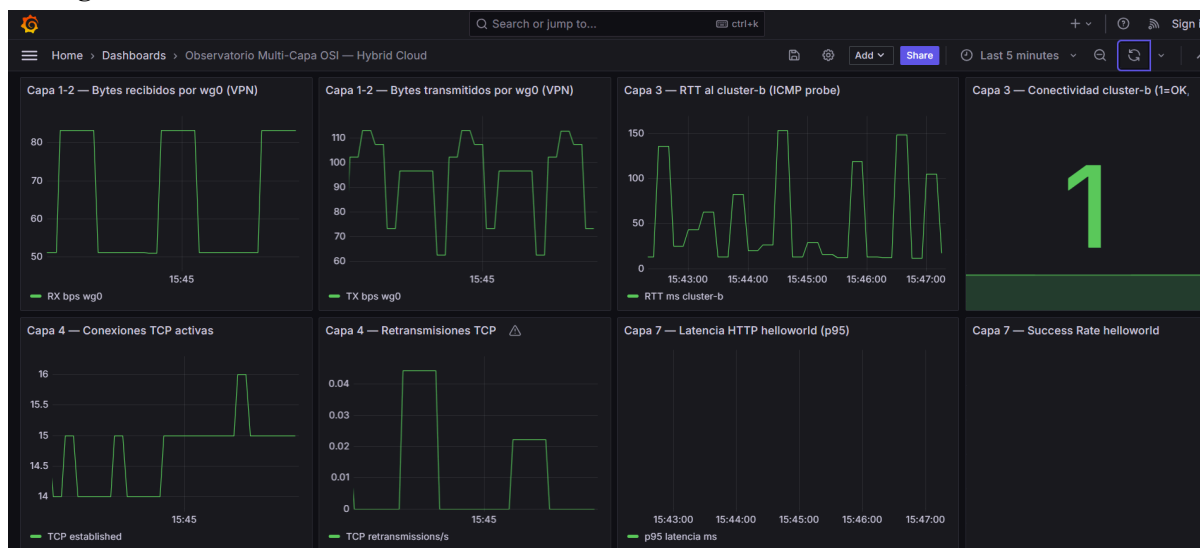
[2/4] Preprocesando datos...
Shape original: (225745, 79)
Clases: {'DDoS': 128027, 'BENIGN': 97718}
✓ 225711 muestras, 20 features
Clases codificadas: {'BENIGN': 0, 'DDoS': 1}

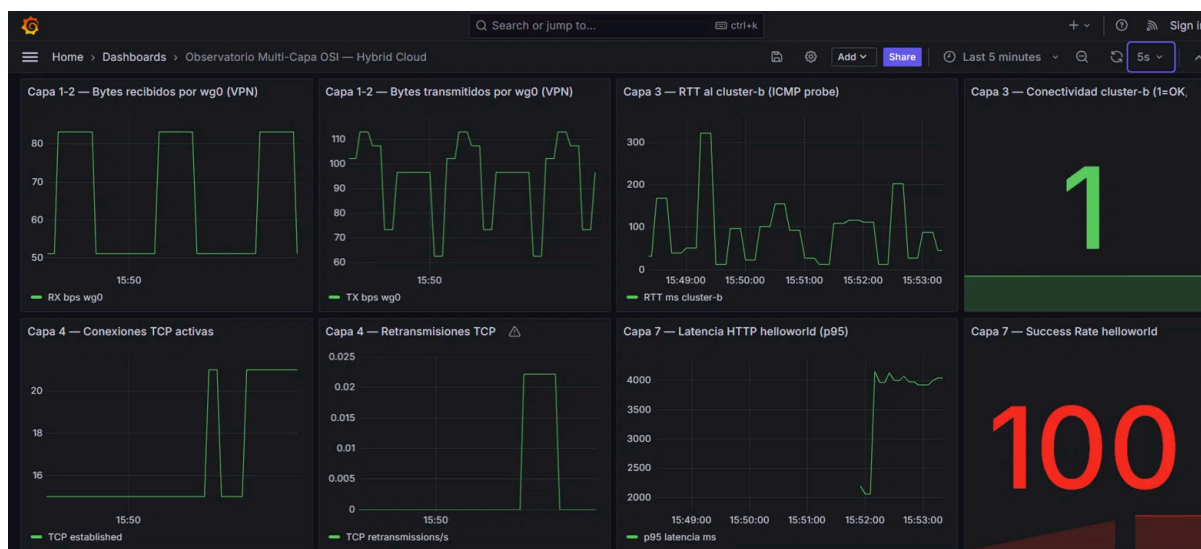
[3/4] Entrenando clasificador Random Forest...
✓ Entrenado en 2.7s
Accuracy: 0.9996 (99.96%)
  
```

2. Pipeline ML sobre VPN

El pipeline descargó 28.4 MB en 10.9 s a través del túnel WireGuard, alcanzando un throughput sostenido de 2.6 MB/s. Procesó 225 745 muestras del archivo de tráfico DDoS, balanceadas entre 128 027 paquetes anómalos y 97 718 benignos. El clasificador Random Forest entrenado on-premises alcanzó una *accuracy* de 99.96 % con precision y recall de 1.00 en ambas clases sobre el conjunto de test.

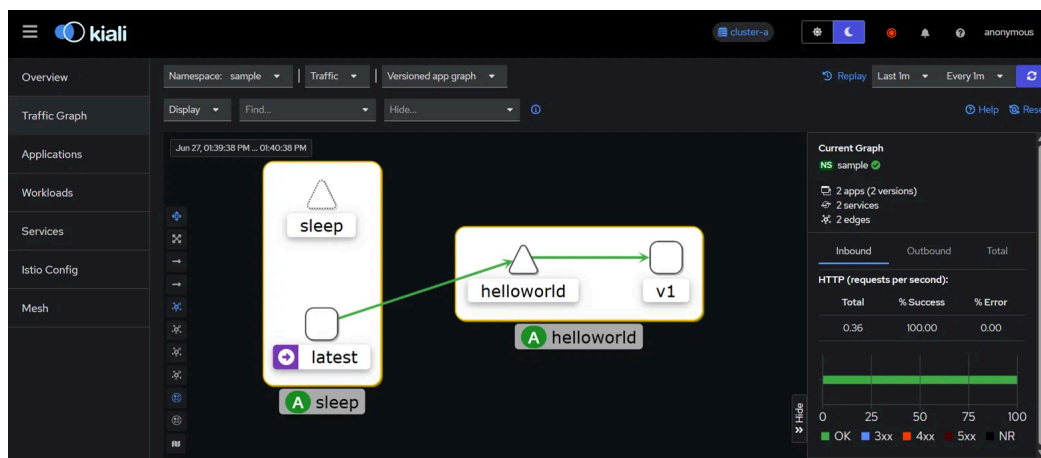
3. Diagnóstico PMTU en el observatorio





Antes de inducir el problema, las cuatro capas del dashboard mostraban valores estables: tráfico continuo en capa 1-2, RTT de ~1 ms en capa 3, conexiones TCP activas en capa 4 sin retransmisiones, y 100 % de success rate en capa 7. Sin embargo, al reducir la MTU del túnel a 576 bytes y bloquear los ICMP, se observaron cambios simultáneos en múltiples capas: el RTT en capa 3 subió de ~1 ms a picos de hasta 300 ms; en capa 4 aparecieron retransmisiones TCP que antes no existían; en capa 1-2 el throughput se mantuvo pero con un patrón más entrecortado. Y tras aplicar MSS clamping con iptables, las retransmisiones en capa 4 cayeron a cero en el siguiente intervalo de scrape y la conectividad en capa 3 se mantuvo. La recuperación fue visible en el dashboard sin necesidad de reiniciar la aplicación.

4. Observabilidad del mesh



Kiali mostró el grafo de tráfico `sleep` → `helloworld` con 100 % de éxito y los candados de mTLS en los enlaces. Grafana también reportó la tasa de peticiones, el success rate del 100 % y las latencias por percentil, con la indicación de mTLS activo en todas las comunicaciones.

Discusión

El overhead del mesh es bajo en operación nominal. Una latencia p95 de 63 ms para tráfico que atraviesa proxies Envoy, mTLS y posible ruteo cross-cluster es un costo razonable frente a los beneficios de seguridad y observabilidad que aporta el mesh.

El mesh aísla el tráfico local de la degradación WAN. El resultado más interesante del experimento 1 es que la degradación del túnel no aumentó la latencia observada. La explicación es que, bajo balanceo con preferencia de localidad (*locality load balancing*), el mesh dirige la mayoría del tráfico al endpoint local y solo recurre al endpoint remoto cuando es necesario. Este comportamiento es deseable en hybrid cloud: la aplicación se mantiene responsiva aun cuando el enlace a la nube se degrada.

El mesh no es solo overhead: también es habilitador de conectividad. El escenario sin mesh, con 100 % de error, ilustra que en una arquitectura cloud-native el service mesh no es un mero agregado sino parte de la infraestructura de comunicación.

Pipeline ML como carga realista. La descarga sostenida de 2.6 MB/s a través del túnel WireGuard demuestra que el enlace soporta cargas de trabajo reales más allá de pequeñas peticiones HTTP. La *accuracy* de 99.96 % del clasificador es alta pero consistente con la literatura sobre CIC-IDS2017: el dataset tiene features muy discriminativas que hacen que clasificadores tipo Random Forest alcancen estos valores. Para los fines de este trabajo, la importancia del pipeline no está en el modelo sino en su rol como carga representativa.

Observabilidad multi-capa como herramienta de diagnóstico. El experimento 3 ilustra el valor de tener un dashboard único que muestre simultáneamente las cinco capas. Un problema en capa 3 (MTU/ICMP) se manifiesta como retransmisiones en capa 4, lo que sin el observatorio sería difícil de correlacionar en vivo. El MSS clamping demuestra además que muchos problemas de PMTU se resuelven más confiablemente *forzando* la solución (ajustando MSS en el handshake) que dependiendo del mecanismo nativo (descubrimiento vía ICMP), porque el ICMP suele ser bloqueado en redes reales.

Limitaciones. La aplicación de `tc netem` sobre una interfaz WireGuard montada sobre WSL no se propaga de forma totalmente fiel, por lo que la degradación efectiva sobre el tráfico cross-cluster fue menor a la nominal. WireGuard, además, maneja la fragmentación a nivel UDP internamente, lo que hace que el black hole clásico (sobre TCP puro sin encapsulación) sea más difícil de reproducir fielmente que en una VPN basada en IPsec/IKEv2 (RFC 7296). El experimento del observatorio igual mostró el síntoma característico (retransmisiones TCP) y la recuperación tras el fix.

Conclusiones

Se construyó una maqueta funcional de una arquitectura Hybrid Cloud que integra varias de las tecnologías de red del curso: una VPN WireGuard como enlace site-to-site entre entornos, dos redes LAN con direccionamiento separado comunicadas sobre una WAN simulada, un service mesh aplicando principios de SDN para gestionar el tráfico entre microservicios, mTLS para la seguridad extremo a extremo, y APIs declarativas para toda la configuración.

Sobre esa maqueta se ejecutó un pipeline de Machine Learning real con datos en Azure Blob Storage, demostrando que el enlace híbrido soporta cargas de trabajo representativas. Se construyó un observatorio multi-capa OSI que permitió diagnosticar en vivo el problema clásico de PMTU black hole y aplicar la mitigación estándar (MSS clamping), validando todo el flujo dentro del mismo dashboard.

El análisis cuantitativo mostró que el overhead del mesh en operación nominal es bajo (p95 de 63 ms con 0 % de error), que el mesh protege el tráfico local de la degradación del enlace WAN, y que el mesh es además un componente habilitador de la conectividad y no un mero agregado.

Las principales dificultades técnicas fueron lograr el mTLS cross-cluster (que requirió una CA compartida correctamente propagada a todos los componentes de ambos clusters) y reproducir fielmente el PMTU black hole sobre WireGuard, que por diseño maneja la fragmentación de manera robusta. Estas experiencias ilustran tanto la fragilidad de la gestión de identidad criptográfica en arquitecturas distribuidas como las decisiones de diseño que distinguen a un protocolo de VPN moderno de los más clásicos.

Referencias

Documentación oficial

1. Istio. *Install Multicluster, Install Primary-Remote*. <https://istio.io/latest/docs/setup/install/multicluster/>
2. Istio. *Plug in CA Certificates*. <https://istio.io/latest/docs/tasks/security/cert-management/plugin-ca-cert/>
3. Istio. *Locality Load Balancing*. <https://istio.io/latest/docs/tasks/traffic-management/locality-load-balancing/>
4. WireGuard. *Conceptual Overview y Protocol & Cryptography*. <https://www.wireguard.com/>
5. Kubernetes. *Cluster Networking, The Kubernetes network model*. <https://kubernetes.io/docs/concepts/cluster-administration/networking/>
6. Envoy Proxy. *Architecture overview*. https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/arch_overview
7. Microsoft Azure. *Blob Storage, REST API and SAS authentication*. <https://learn.microsoft.com/azure/storage/blobs/>
8. Microsoft Azure. *VPN Gateway documentation*. <https://learn.microsoft.com/azure/vpn-gateway/>
9. Prometheus. *Documentation, Configuration*. <https://prometheus.io/docs/prometheus/latest/configuration/>
10. Grafana k6. *Documentation*. <https://grafana.com/docs/k6/latest/>
11. Linux `tc-netem(8)` y `iptables(8)` manual pages. <https://man7.org/linux/man-pages/>

RFCs

12. RFC 791 - *Internet Protocol*. <https://www.rfc-editor.org/rfc/rfc791>
13. RFC 792 - *Internet Control Message Protocol*. <https://www.rfc-editor.org/rfc/rfc792>
14. RFC 1191 - *Path MTU Discovery*. <https://www.rfc-editor.org/rfc/rfc1191>
15. RFC 2923 - *TCP Problems with Path MTU Discovery*. <https://www.rfc-editor.org/rfc/rfc2923>

16. RFC 4821 - *Packetization Layer Path MTU Discovery*. <https://www.rfc-editor.org/rfc/rfc4821>
17. RFC 4364 - *BGP/MPLS IP Virtual Private Networks*. <https://www.rfc-editor.org/rfc/rfc4364>
18. RFC 7296 - *Internet Key Exchange Protocol Version 2 (IKEv2)*.
<https://www.rfc-editor.org/rfc/rfc7296>

Dataset

19. Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). *Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization*. Canadian Institute for Cybersecurity. <https://www.unb.ca/cic/datasets/ids-2017.html>

Anexos

Los siguientes materiales se incluyen como anexos y no cuentan dentro del límite de páginas del informe.

- Repositorio del proyecto: <https://github.com/aleaurre/redato-hybrid-cloud>.
Contiene las configuraciones de Istio multi-primario ([mesh/istio-a.yaml](#), [mesh/istio-b.yaml](#)), las políticas de tráfico ([mesh/policies/](#): canary, circuit breaker y retries), el pipeline de ML ([pipeline/csv_to_parquet.py](#), [pipeline/ml_pipeline.py](#)), y los scripts de carga y análisis de los experimentos ([experiments/load-test.js](#), [experiments/analyze.py](#), [experiments/run-all.sh](#)).
 - **Anexo A, Latencia comparativa (k6):** gráfica de latencia entre los tres escenarios evaluados (con mesh baseline, con mesh sobre WAN degradada, y sin mesh) ([experiments/results/latencia-comparacion.png](#)). Los resultados crudos de cada corrida están en [experiments/results/*.json](#).
 - **Anexo B, Grafo de tráfico con mTLS (Kiali):** capturas del grafo de servicios mostrando mTLS activo entre clústeres ([capturas/kiali1.png](#), [capturas/kiali2.png](#)).
 - **Anexo C, Métricas de observabilidad (Grafana):** capturas del Istio Service Dashboard con success rate y latencia por percentil ([capturas/graphana1.png](#) a [graphana3.png](#)), y del Observatorio Multi-Capa OSI mostrando las cuatro capas simultáneamente ([capturas/graphanaMultiCapa.png](#) y [demo/graphanamulti1.png](#) a [graphanamulti5.png](#)).
 - **Anexo D, Resultados estadísticos:** tabla resumen con media, mediana, p95 y desvío estándar de latencia por escenario ([experiments/results/summary.csv](#)).